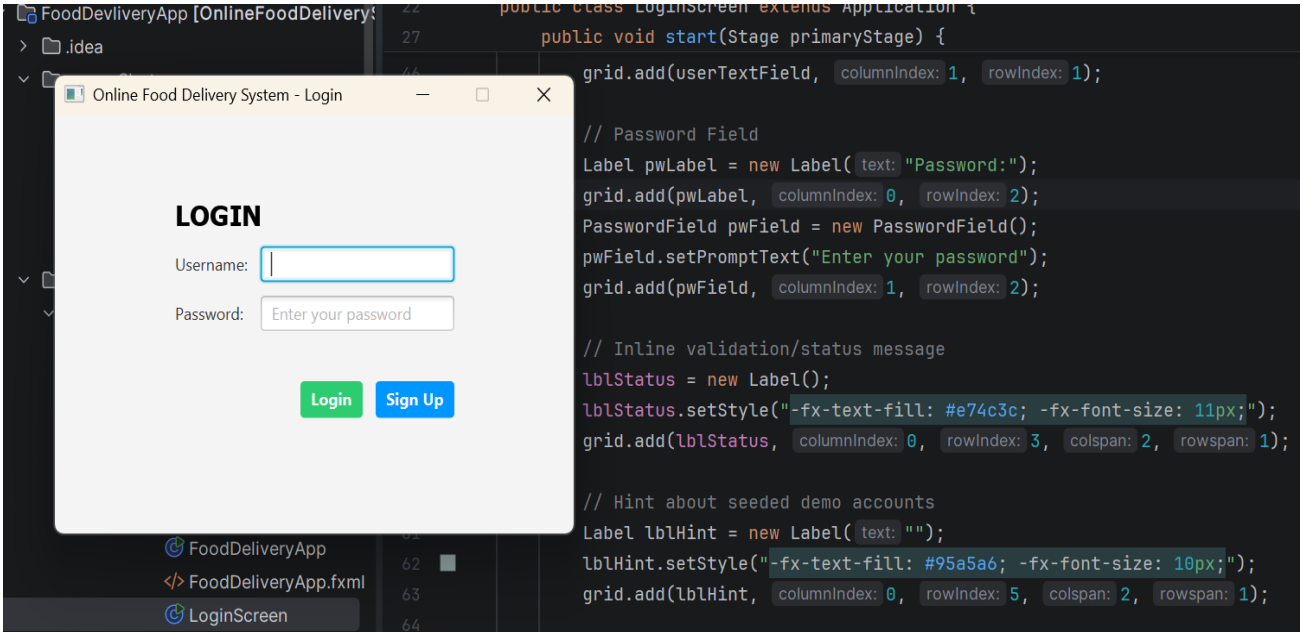
NAME: PRINCE PIO ANAAYIRE INDEX NUMBER: MS/ITE/25/0037

TECHNICAL REPORT

ONLINE FOOD DELIVERY SYSTEM



Demo Login Credentials		
	username	password
Admin	prince	prince123
Customer	linda	Linda123

Project Detail	
Project Title	Online Food Delivery System (FoodDevliveryApp)
Application Type	Java desktop application (JavaFX)
Repository	**https://github.com/PRINCE2025EASY/FoodDevliveryApp**
Report Date	July 11, 2026

2. **Introduction**

The Online Food Delivery System is a desktop application built with Java and JavaFX that simulates the core workflow of a restaurant ordering platform: a customer browses a menu, builds a shopping cart, and submits an order that is captured as a formal invoice; a separate administrator role can subsequently review every order placed across all customer sessions from a dedicated dashboard. The project is packaged as a Maven module named **Online FoodDelivery System** and organised around a small, purpose-built object model rather than a database-backed enterprise stack, making it well suited as a teaching example of desktop GUI development, role-based navigation, and basic object-oriented design in Java.

Unlike typical class exercises that stop at a single ordering screen, this project extends the brief with two connected concerns that mirror real commercial delivery platforms:

authentication with distinct account roles **(customer vs. administrator)**, and a shared, persistent order ledger that both roles can observe.

A login gate built on an in-memory AuthService routes a successful sign-in to one of two JavaFX scenes:

1. **FoodDeliveryApp** for customers.
2. **AdminDashboard** for administrators.

While a singleton OrderStore keeps every placed order synchronised in memory and mirrors it to a flat text file on disk so that orders survive between application runs.

The problem it addresses, its design and class structure, the object-oriented principles it demonstrates, the graphical interface it presents to each type of user, and the practical challenges encountered while building and evaluating it.

3. Problem Statement

Manual or informally-managed food ordering telephone orders, walk-in orders recorded on paper, or ad-hoc messaging creates friction for both customers and restaurant operators. Customers cannot see a structured, itemised menu with prices at the point of ordering, and are prone to communication errors around delivery details.

Restaurant staff have no consolidated, searchable record of orders placed across different shifts or communication channels, which makes it hard to reconcile revenue, verify delivery addresses, or resolve customer disputes after the fact.

The Online Food Delivery System addresses a scoped version of this problem for a single-restaurant scenario: it needs to

(a) let an authenticated customer browse a fixed menu, assemble a cart, and submit an order with delivery details, producing an immediate, itemised invoice; and

(b) give a separate, authenticated administrator a single place to see every order placed by every customer, search/filter that list, and inspect the full invoice for any individual order, together with a running total of orders and revenue.

4. Objectives of the System

The system was designed to satisfy the following objectives:

1. Provide a simple, role-aware login mechanism that distinguishes between **customer** and **administrator** accounts and routes each to an appropriate screen.
2. Allow a customer to view an itemised restaurant menu, add items to a shopping cart, and see a running total as items are added.
3. Validate customer checkout details (name, email, delivery address) before an order can be finalised, and reject incomplete or malformed submissions with clear feedback.
4. Generate a structured invoice for every placed order and persist it so it is not lost when the application restarts.
5. Give an administrator a consolidated, searchable view of every order placed by every customer, along with aggregate metrics (order count and total revenue).
6. Demonstrate core object-oriented programming concepts abstraction, encapsulation, inheritance, and polymorphism in a coherent, working domain model rather than as isolated syntax examples.

7. Keep the codebase small enough to be read and understood end-to-end, favouring an in-memory/flat-file approach over external infrastructure such as a relational database or web server.

5. Scope of the System

5.1 In Scope

- Username/password authentication against an in-memory credential store, with two roles: ADMIN and CUSTOMER.
- A self-service sign-up dialog that lets a new user register as either a customer or an administrator account.
- A customer ordering screen: menu listing, add-to-cart, running total, checkout form, input validation, and an on-screen order confirmation/invoice.
- A shared order store that persists every placed order to a flat text file (orders_data.txt) and reloads it on startup, so orders are visible across sessions.
- An administrator dashboard: a searchable/filterable table of all orders, a detail pane showing the full invoice for the selected order, and live order-count/revenue summary statistics.
- Logout from either screen back to the login screen.

5.2 Out of Scope

- Real payment processing or integration with a payment gateway.
- Live GPS delivery tracking, courier assignment, or notifications.
- A relational database, ORM, or network-hosted backend — persistence is a local flat file, and credentials are memory-only (they reset on restart).
- Menu management UI for administrators the current menu (five items) is seeded in code (loadMockMenuData()) rather than editable through the interface.
- Multi-restaurant or multi-tenant support the system models a single restaurant ("Prince Online Food Delivery Company").

6. Methodology

The system was developed as a single-module Java project using an iterative, feature-by-feature approach typical of small desktop applications:

6.1 Technology Stack

Layer	Technology
Language	Java (source/target level 9, per pom.xml compiler configuration)
GUI Framework	JavaFX 21 (javafx-controls, javafx-fxml)
Build Tool	javafx-maven-plugin for packaging/launch
Auxiliary Libraries	ControlsFX 11.2.1, FormsFX 11.6.0 (declared as dependencies)
Testing	JUnit Jupiter 5.12.1 (declared as a test-scope dependency)
Persistence	Flat text file (orders_data.txt) via java.nio.file; no external database
Module System	Java Platform Module System (module-info.java)

6.2 Development Approach

Development followed a layered, incremental build-out:

8. **Domain modelling first:** the core classes User, Customer, MenuItem, and Order were defined to represent the vocabulary of the problem before any UI code was written.
9. **A single-screen ordering prototype (FoodDeliveryApp**) was built directly on top of the domain model, using JavaFX's programmatic (code-based) UI construction rather than FXML for the main screens.

10. **Authentication and role-based** routing were layered in afterwards via AuthService and LoginScreen, turning the single-screen prototype into a two-role application.
11. **A shared OrderStore** singleton was introduced to decouple order creation (in **FoodDeliveryApp)** from order consumption (in the new AdminDashboard), using JavaFX's ObservableList so the admin table updates live and flat-file persistence so history survives a restart.
12. **The AdminDashboard** was added last, reusing the same Order and OrderStore abstractions rather than duplicating order-handling logic.

This mirrors a bottom-up methodology: stable domain classes at the base, a working vertical slice (ordering) built on top of them, then breadth added (authentication, roles, an administrative view) without having to rewrite the foundation.

6.3 Design Principles Applied

- Single Responsibility: each class has one clear job — MenuItem is a value object, OrderStore only manages persistence and the observable order list, LoginScreen only handles authentication UI and routing.
- Singleton pattern for OrderStore, ensuring the customer screen and the admin screen always observe the same in-memory order list.

- Immutable snapshots for Order once created, an order's items and totals do not change, even if the underlying MenuItem catalogue is edited later, which protects historical invoice accuracy.
- Separation of UI and domain logic MenuItem, Order, Customer, User, AuthService, and OrderStore contain no JavaFX imports; only the three Application subclasses do.

7. System Design

The system follows a straightforward layered structure: a domain/model layer (User, Customer, MenuItem, Order), a service/persistence layer (AuthService, OrderStore), and a presentation layer made up of three JavaFX Application subclasses (LoginScreen, FoodDeliveryApp, AdminDashboard) that hand the same primaryStage off to one another as the user navigates.

7.1 High-Level Architecture and Application Flow

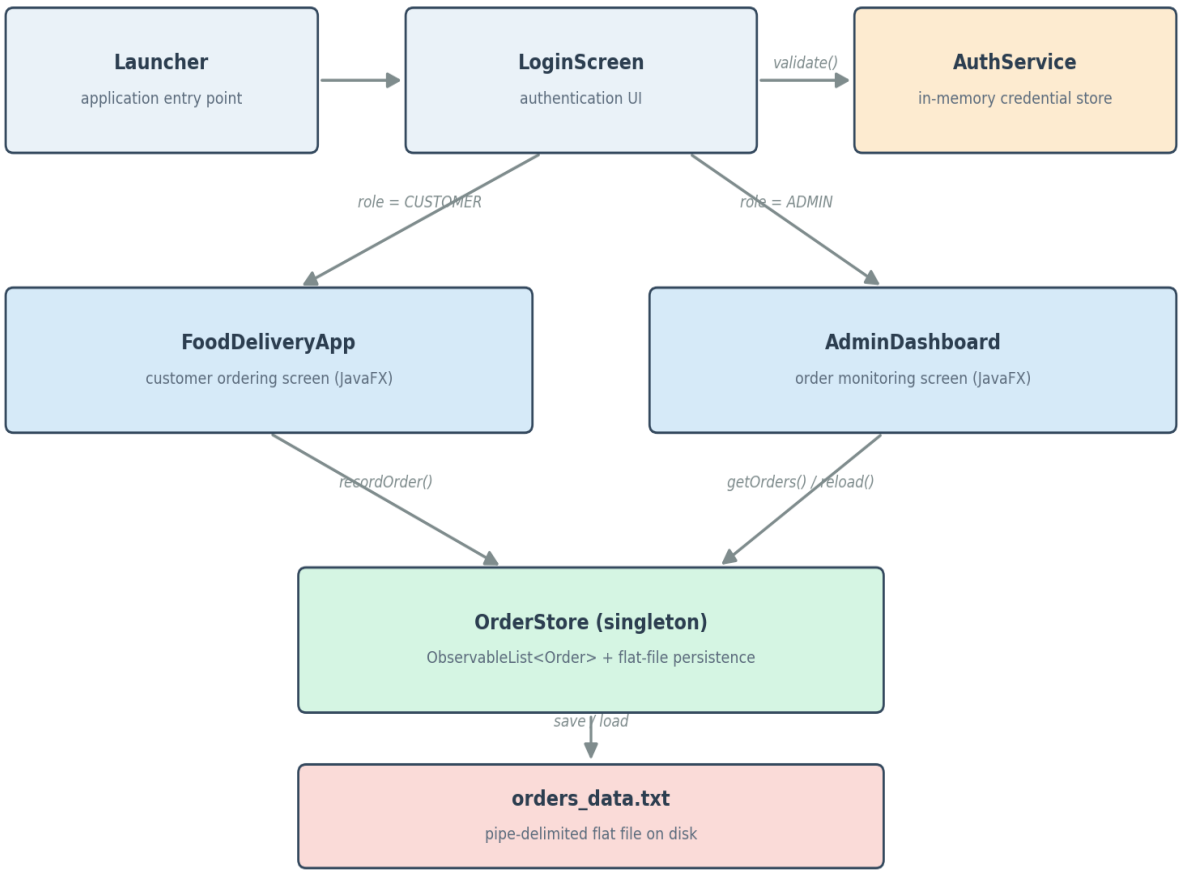


Figure 1: Application flow from launch, through role-based routing, to shared order persistence.

Launcher.main() starts the JavaFX runtime on LoginScreen. A successful authentication check in AuthService determines the account's Role; LoginScreen then constructs either a new FoodDeliveryApp or a new AdminDashboard and calls start(primaryStage) on it directly, reusing the same window rather than opening a second one. Both of the resulting screens read from and write to the single OrderStore instance, which is the only component that touches the disk.

7.2 Class Diagram

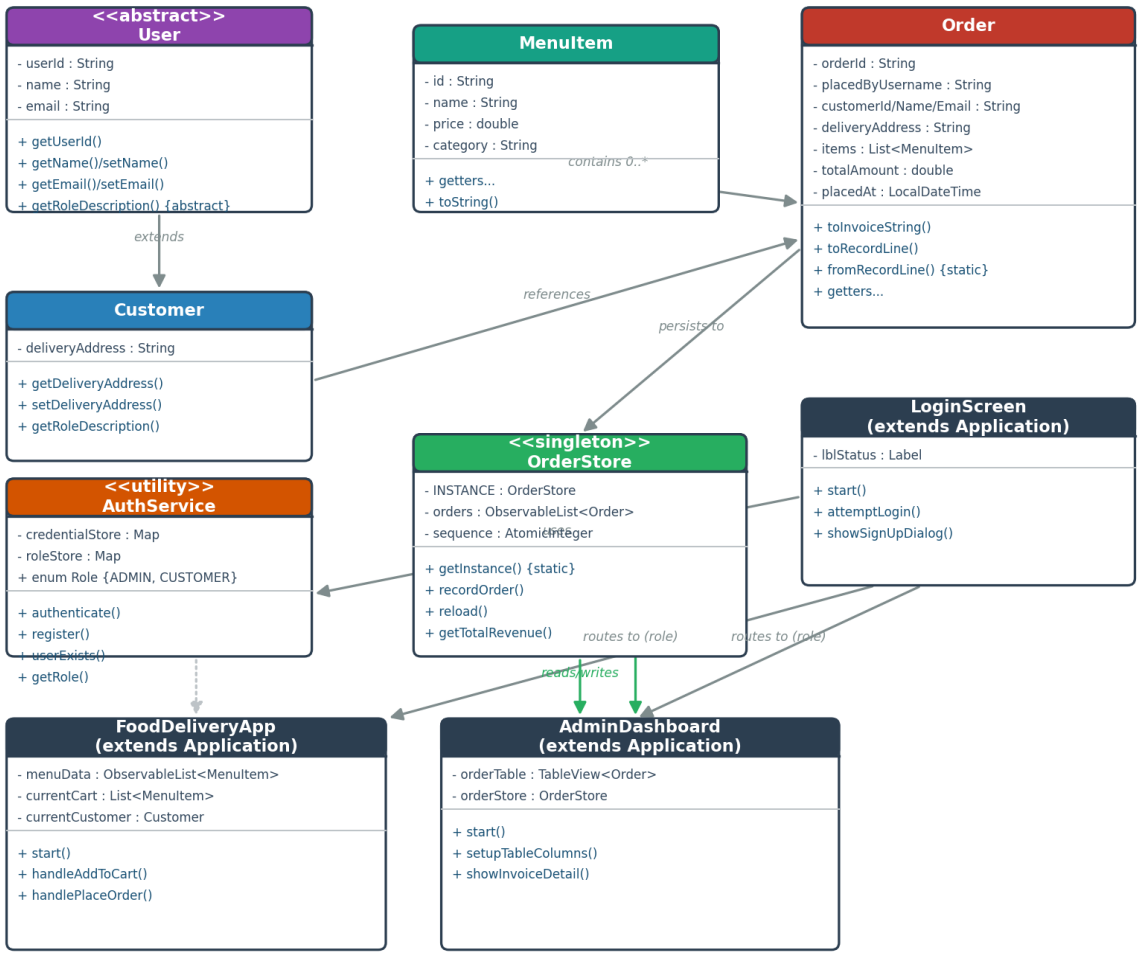


Figure 2: Class relationships across the domain, service, and presentation layers.

7.3 Package Structure

The source tree contains two Java packages, reflecting the project's evolution from a generated JavaFX template into a purpose-built application:

- com.delivery — the application proper: AuthService, User, Customer, MenuItem, Order, OrderStore, LoginScreen, FoodDeliveryApp, and AdminDashboard.
- com.example.onlinefooddeliverysystem — the original JavaFX "Hello World" scaffold (HelloApplication, HelloController, hello-view.fxml) generated when the JavaFx project was first created, plus Launcher, the class actually referenced by the Maven run configuration and by module-info.java as the application's entry point.

module-info.java declares the module com.example.onlinefooddeliverysystem, requires javafx.controls, javafx.fxml, and javafx.base, and opens both packages to the JavaFX runtime — notably opens com.delivery to javafx.graphics (needed to launch the Application subclasses that live there) and to javafx.base (needed for PropertyValueFactory's reflective access to Order's getters when populating the AdminDashboard's TableView).

7.4 Data Flow: Placing and Viewing an Order

13. The customer selects a MenuItem in the menu ListView and clicks "Add Selected Item to Cart"; handleAddToCart() appends it to an in-memory List<MenuItem> and updates the running total label.
14. On "Finalize & Process Delivery Order", handlePlaceOrder() validates the cart and the name/email/address fields, then constructs a new Customer object from the form input.
15. OrderStore.recordOrder() generates a sequential order ID (ORD-0001, ORD-0002, …), wraps the customer, cart, and total into an immutable Order, inserts it at the head of the shared ObservableList<Order>, and writes the full order list back to orders_data.txt.
16. The same Order's toInvoiceString() is rendered into the customer's on-screen ledger/console area as confirmation.
17. Whenever an administrator opens or refreshes AdminDashboard, it calls OrderStore.reload(), which re-reads orders_data.txt from disk — so an admin session started after the customer's order will still see it, even in a fresh application run.
18. Selecting a row in the admin order table triggers showInvoiceDetail(), which renders that Order's toInvoiceString() into a read-only detail pane.

8. Description of Classes and Methods

This section summarises the responsibility and key members of every class in the com.delivery package, which contains all of the application's meaningful logic.

8.1 User (abstract)

An abstract base class representing any account holder. Declares the shared identity fields (userId, name, email) and forces subclasses to describe their own role.

Member	Type	Description
userId	field (final)	Randomly generated 8-character identifier (from UUID), assigned once in the constructor and never exposed via a setter — an encapsulation choice that keeps identity immutable.
name, email	fields	Mutable profile data, exposed through conventional getters and setters.
getRoleDescription()	abstract method	Forces every concrete subclass to state, in its own words, what the role can do; overridden polymorphically by Customer.

8.2 Customer extends User

Represents the person placing an order. Adds a deliveryAddress field on top of the inherited identity fields and overrides getRoleDescription() to describe customer permissions ("Allowed to view menus, place orders, and track deliveries."). A new Customer is constructed at checkout time in FoodDeliveryApp from the values typed into the name/email/address fields.

8.3 MenuItem

A plain value object describing one item on the restaurant's menu: an id, a name, a price (double), and a category. It overrides toString() to render as "Name (Category) - $Price", which is what the JavaFX ListView displays for each menu row without any custom cell factory.

8.4 Order

An immutable snapshot of a completed transaction — the most detailed class in the system. It stores identifying information (orderId, placedByUsername, customer details), the list of MenuItem objects purchased, the totalAmount, and a placedAt timestamp.

Method	Description
toInvoiceString()	Builds the full, human-readable invoice text (order ID, customer/delivery details, itemised list, total, status) shown both to the customer at checkout and to the administrator in the dashboard.
toRecordLine() / fromRecordLine()	Serialise and deserialise an Order to/from a single pipe-delimited line, used by OrderStore for flat-file persistence. Menu items within a line are further encoded with '~' and ';' separators; a private escape() helper strips any of these delimiter characters out of free-text fields (e.g. the delivery address) so a line can never be corrupted by user-entered punctuation.
getFormattedTimestamp()	Formats placedAt using the pattern yyyy-MM-dd HH:mm:ss for consistent display in both the customer ledger and the admin table.

8.5 AuthService (final, static utility)

A self-contained authentication service backed entirely by two in-memory ConcurrentHashMap instances — one mapping username to password, the other username to Role. It seeds two demo accounts on class load (admin/admin123 as ADMIN, customer/customer123 as CUSTOMER).

Method	Description
authenticate(username, password)	Returns true only if the username exists and the stored password matches exactly (plain-text comparison).
register(username, password[, role])	Adds a new account using putIfAbsent so a username cannot be silently overwritten; returns false if the username is already taken or either field is blank.
userExists(username)	Used by the sign-up dialog to pre-empt duplicate registrations with a friendly message.
getRole(username)	Looks up the account's Role, defaulting to CUSTOMER for any unrecognised username.

8.6 OrderStore (final, singleton)

The single shared source of truth for every order in the running application, exposed only through the static getInstance() accessor (private constructor). It holds an ObservableList<Order> so JavaFX table/list views bound to it update automatically, plus an AtomicInteger sequence used to generate collision-free order IDs across concurrent-feeling access.

Method	Description
recordOrder(username, customer, items, total)	Synchronized. Builds the next order ID, constructs the Order, inserts it at index 0 (newest-first), and immediately persists the whole list to disk.
reload()	Clears and re-reads the observable list from orders_data.txt; used on AdminDashboard startup and its "Refresh" button.
getTotalRevenue()	Sums totalAmount across every currently loaded order, feeding the dashboard's summary label.
loadFromDisk() / saveToDisk()	Private helpers wrapping java.nio.file.Files read/write calls, with I/O exceptions caught and logged rather than propagated, so a persistence failure does not crash the UI.

8.7 LoginScreen extends Application

The application's entry screen. Builds a GridPane login form (username, password, status label, Login/Sign Up buttons) and a modal sign-up dialog. attemptLogin() validates that both fields are non-empty, delegates to AuthService.authenticate(), and on success reads the account's Role via AuthService.getRole() to decide whether to call openFoodDeliveryApp() or openAdminDashboard(), both of which hand the existing Stage to the next screen's start() method rather than opening a new window.

8.8 FoodDeliveryApp extends Application

The customer-facing ordering screen. Maintains an ObservableList<MenuItem> for the menu, a plain List<MenuItem> for the active cart, and a running cartTotal. Key private methods: loadMockMenuData() seeds five sample dishes; handleAddToCart() moves the selected menu row into the cart and updates the total label; handlePlaceOrder() performs input validation (non-empty cart, non-empty name/email/address, a basic '@'/'.' check on the email) inside a try/catch that distinguishes expected validation

failures (IllegalArgumentException, IllegalStateException) from any other unexpected exception, before delegating persistence to OrderStore.recordOrder() and clearing the cart via clearCartSession().

8.9 AdminDashboard extends Application

The administrator's monitoring screen. Wraps OrderStore.getOrders() in a FilteredList<Order> driven by a search TextField, so typing into the filter box narrows the TableView by order ID, customer name, email, or the username that placed the order all without touching the underlying data. setupTableColumns() wires six TableColumn<Order, ?> instances using PropertyValueFactory reflection (for direct fields) and inline cell-value factories (for the formatted currency and timestamp columns). Selecting a row calls showInvoiceDetail() to render that order's full invoice text; a ListChangeListener on the underlying order list keeps the order-count/revenue summary label continuously in sync.

8.10 Supporting entry-point classes

Launcher (package com.example.onlinefooddeliverysystem) is the class actually named in module-info.java and the Maven javafx-maven-plugin configuration; its main() calls Application.launch(LoginScreen.class, args), which is what makes the application start on the login screen rather than on the generated HelloApplication template. HelloApplication and HelloController are unused remnants of the default JavaFX

Maven archetype and are not part of the delivery workflow (see Section 13, Challenges Encountered, for further discussion).

9. GUI Design Explanation

All three main screens are built programmatically with JavaFX layout panes (BorderPane, GridPane, VBox, HBox, SplitPane) rather than FXML, which keeps each screen's Java class as the single source of truth for both its structure and its event wiring. A consistent visual language is used throughout: a soft blue-gray background (#f4f6f9), a dark slate heading colour (#2c3e50), and colour-coded action buttons — green (#2ecc71) for confirming/positive actions, blue (#3498db) for neutral/primary actions, and red (#e74c3c) for destructive or exit actions such as Logout.

9.1 Login Screen

LoginScreen lays out a centred GridPane (400×300) with username and password fields, a red inline status label for validation errors, a hint reminding evaluators of the two seeded demo accounts, and Login/Sign Up buttons. The Sign Up button opens a separate modal Stage (Modality.WINDOW_MODAL) so registration cannot be dismissed accidentally mid-entry, and lets the registrant choose an account Role from a ChoiceBox.

9.2 Customer Ordering Screen

FoodDeliveryApp uses a BorderPane root: a top banner with the company title, a welcome message, and a Logout button; a two-column GridPane in the centre (menu ListView + "Add to Cart" button on the left, cart ListView + running total + checkout form + "Finalize" button on the right); and a bottom TextArea acting as a running order/invoice console. This mirrors a familiar e-commerce layout — browse on the left, cart and checkout on the right so the interaction pattern needs no explanation for a first-time user.

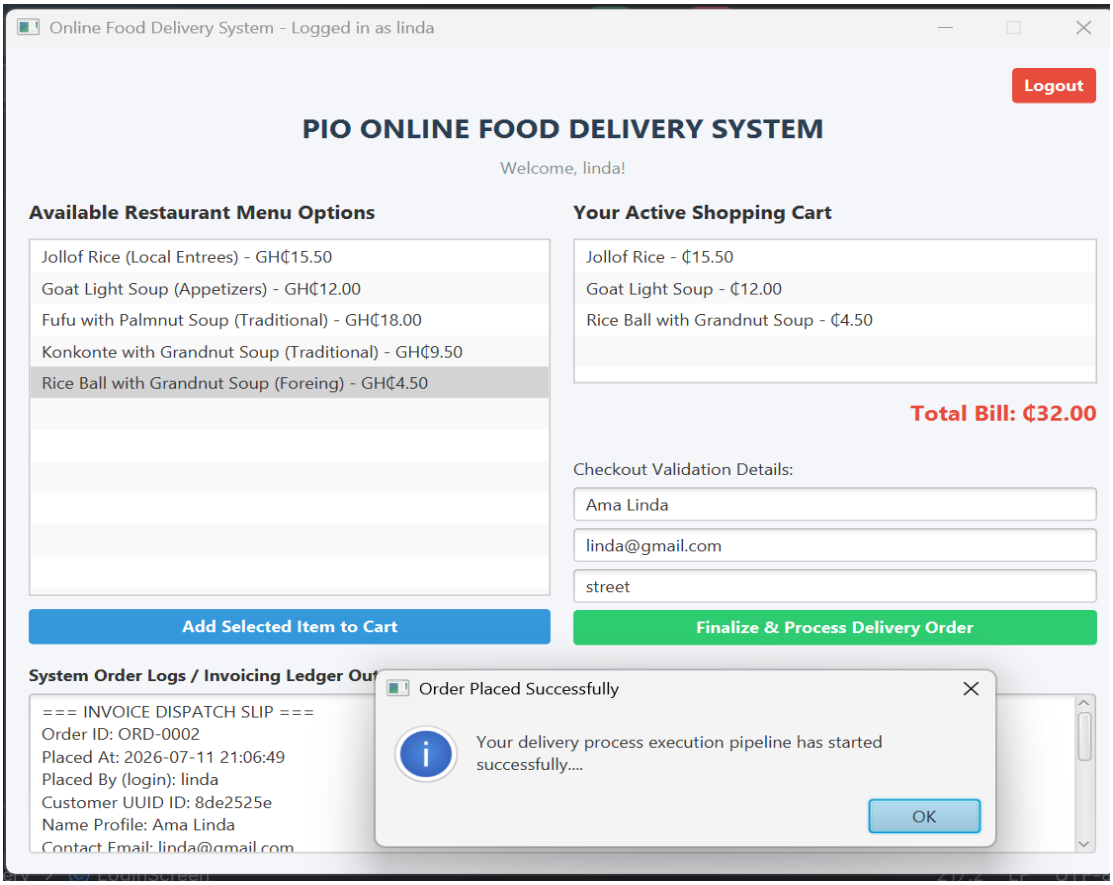


Figure 3: Customer ordering screen layout

9.3 Administrator Dashboard

AdminDashboard also uses a BorderPane root, with a SplitPane in the centre dividing the window roughly 58/42: the left pane holds a search field, a Refresh button, the six-column order TableView, and a live summary label; the right pane holds a read-only TextArea that displays the full invoice for whichever order is currently selected. This master-detail layout lets an administrator scan many orders at once while still being able to drill into the exact wording of any single invoice.

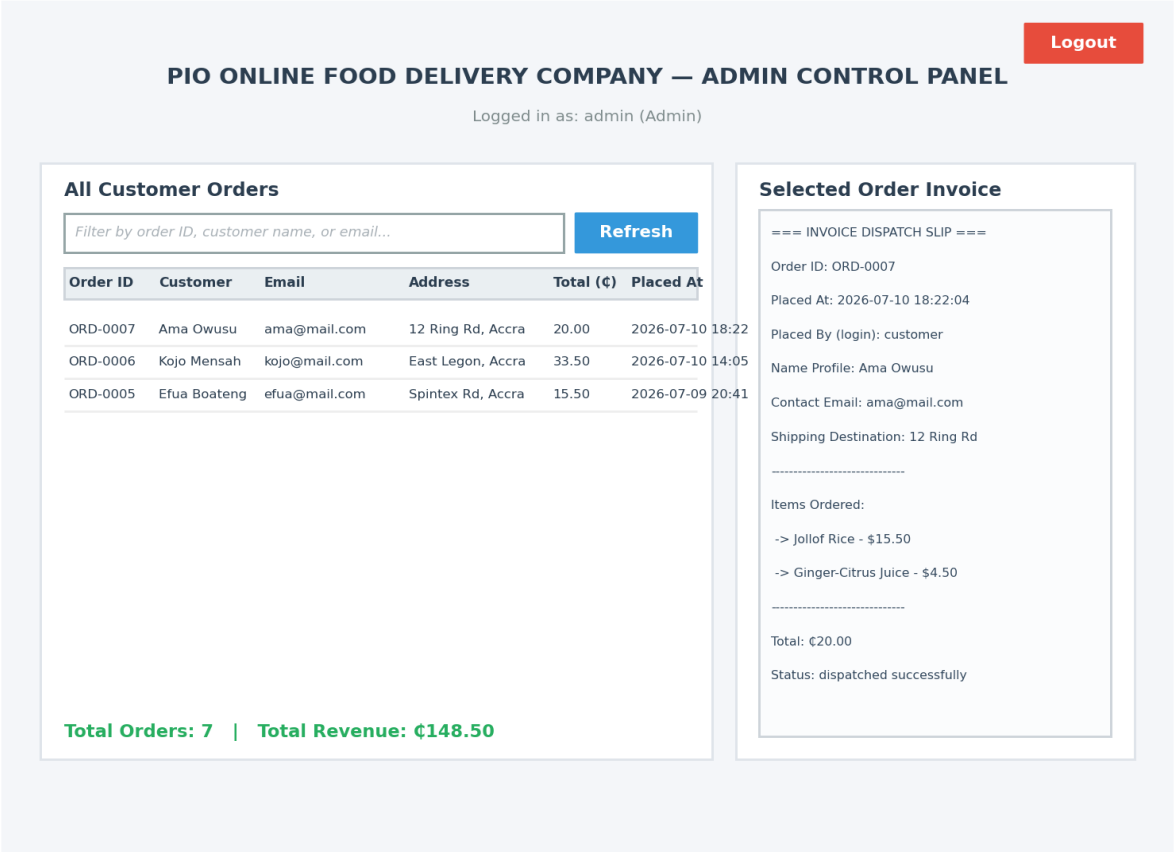


Figure 4: Administrator dashboard layout

9.4 Feedback and Error Handling in the UI

Both the customer and admin screens rely on JavaFX Alert dialogs (Alert.AlertType.WARNING/INFORMATION/ERROR) to surface validation problems and confirmations, so a user is never left wondering whether an action succeeded — for example, an empty cart at checkout produces a clearly worded warning rather than a silent no-op, and a successful order shows an information dialog quoting the new order ID.

10. OOP Concepts Implemented

The domain model was deliberately written to showcase the four pillars of object-oriented programming inside a single, coherent feature set rather than as disconnected examples.

10.1 Abstraction

User is declared abstract and defines the abstract method getRoleDescription(), establishing a contract that any account type must fulfil without dictating how. Client code (and the UI) only ever needs to know that a User can describe its own role — not how that description is produced.

10.2 Encapsulation

Fields across the domain classes are private, with access mediated through explicit getters/setters (e.g. User.getUserId() has no matching setter, making identity effectively read-only after construction). AuthService goes further, keeping its two credential maps private and static with no external code able to reach them except through the class's own authenticate/register/getRole methods — the maps themselves are never returned or exposed.

10.3 Inheritance

Customer extends User, inheriting userId, name, and email plus their accessors, and adding only what is specific to a customer (deliveryAddress). The source code explicitly comments this relationship ("Inheritance: Customer inherits properties and behaviors from User"), and both FoodDeliveryApp and AdminDashboard extend javafx.application.Application to inherit the JavaFX lifecycle (start(), launch()).

10.4 Polymorphism

Customer.getRoleDescription() overrides User's abstract method, so code that only holds a User reference still invokes Customer-specific behaviour at runtime. MenuItem.toString() is overridden so JavaFX's default ListView cell rendering automatically displays a formatted "Name (Category) - $Price" string with no custom cell factory required. Order.toString() and Order.toInvoiceString() likewise override/extend default behaviour to produce two different textual representations (a short summary line and a full invoice) of the same object.

10.5 Additional Design Patterns

- Singleton — OrderStore.getInstance() guarantees exactly one shared order list for the whole application, which is what allows the admin dashboard to see orders placed through the customer screen.

- Static factory / round-trip serialisation — Order.fromRecordLine() acts as a static factory that reconstructs an Order from a persisted line, paired with the instance method toRecordLine() that produced it, keeping the encode/decode logic co-located on the class it belongs to.
- Observer (via JavaFX bindings) — ObservableList<Order> and its ListChangeListener let AdminDashboard's summary label and FilteredList react automatically to changes in OrderStore, rather than being manually refreshed.

11. Screenshots and Outputs

The images below are wireframe reconstructions of the three JavaFX screens, laid out to scale from the actual panel structure, control types, and text found in the application's source code (LoginScreen.java, FoodDeliveryApp.java, and AdminDashboard.java). They are included in place of live screen captures because rendering a JavaFX Stage requires a graphical display and a full Maven dependency download (JavaFX 21, ControlsFX, FormsFX), neither of which is available in the sandboxed, network-isolated environment used to prepare this report. Every control, label, colour, and piece of sample data shown below is taken directly from the corresponding Java class, so the wireframes are a faithful preview of what running mvn clean javafx:run would produce.

11.1 Login Screen

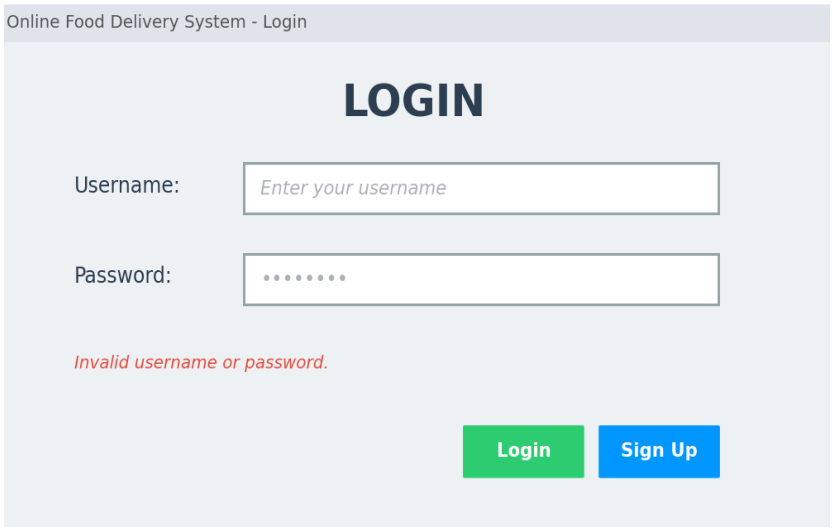


Figure 5: Login screen username/password fields, inline validation status, and the Login / Sign Up actions.

11.2 Customer Ordering Screen

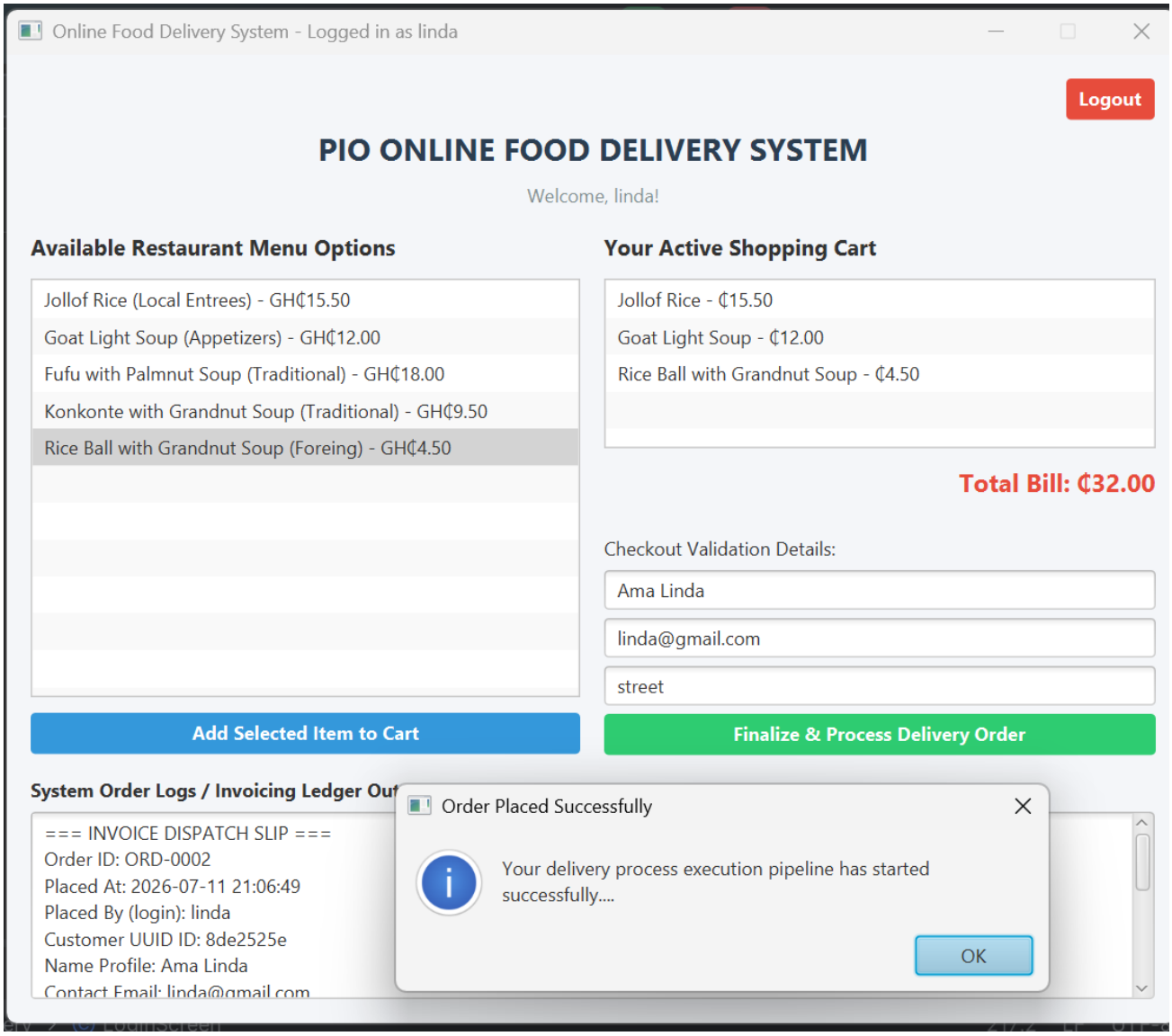


Figure 6: Customer ordering screen after three items have been added to the cart, ready for checkout.

11.3 Administrator Dashboard

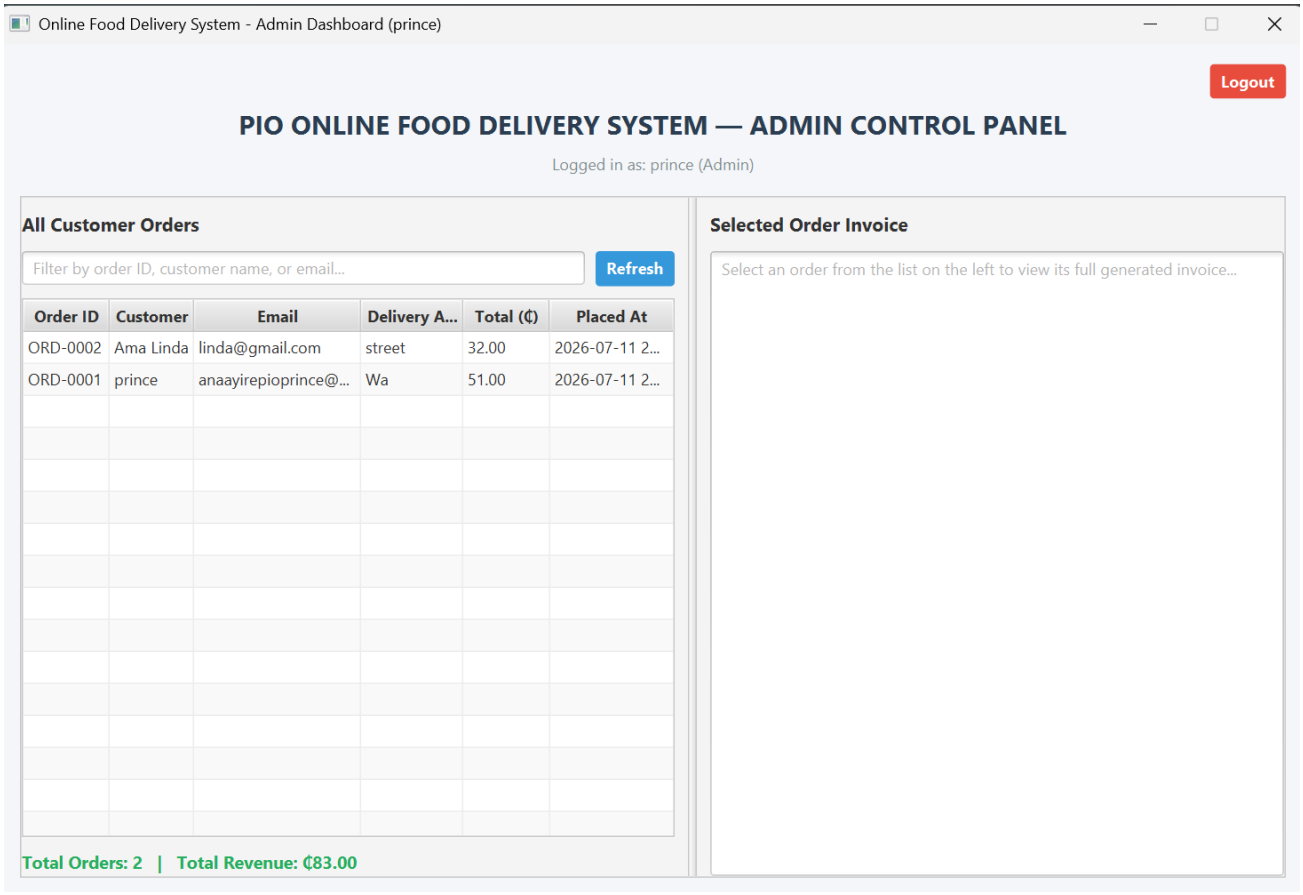


Figure 7: Administrator dashboard showing the consolidated order table, live revenue summary, and a selected order's invoice.

12. GitHub Repository Link

The full source code for this project is hosted at:

https://github.com/PRINCE2025EASY/FoodDevliveryApp

The repository contains the complete JavaFx project (pom.xml), the two Java packages described in Section 7.3, the JavaFX module descriptor (module-info.java), and IDE project metadata. Cloning the repository and running mvn clean javafx:run from the project root will build and launch the application starting at the login screen.

13. Challenges Encountered

Analysing and documenting this codebase surfaced a number of genuine engineering challenges, some resolved cleanly in the code and some left as open issues worth flagging.

13.1 Synchronising two independent views of the same data

Because customer orders and the admin view run as separate screens (potentially in separate application launches), the project had to solve the classic problem of two UI surfaces needing a consistent view of the same mutable data. The chosen solution a singleton OrderStore backed by an ObservableList and mirrored to a flat file keeps the in-memory case simple (JavaFX bindings propagate changes automatically within one running instance) while still covering the cross-session case via reload()/loadFromDisk(). The trade-off is that concurrent writes are not truly transactional: saveToDisk() rewrites the entire file on every recordOrder() call, which is safe for a single-user demo but would not scale to genuinely concurrent multi-user access.

13.2 Keeping invoices accurate as the menu changes

Because MenuItem objects are mutable value objects with no independent identity beyond their id/name/price fields, care had to be taken so that a later change to the seeded menu could never retroactively rewrite the price shown on an already-placed order. The project addresses this by having Order store its own independent copy of each purchased MenuItem (and re-serialising that exact snapshot to disk), rather than storing references back into the live menu list — an example of designing for data integrity over convenience.

13.3 Flat-file persistence and its limits

Choosing a hand-rolled, pipe-delimited flat file over an embedded database (e.g. SQLite) kept the project dependency-free and easy to inspect, but it pushed the burden of correct escaping onto the application: delivery addresses, names, and other free-text fields could in principle contain the same **'|', '~', or ';'** characters used as field/item separators. The Order.escape() helper defends against this by stripping those characters from user-entered text before it is written, which is a pragmatic fix but does mean that, for example, a pipe character typed into a delivery address is silently converted to a forward slash rather than preserved exactly.

13.4 Plain-text credential storage

AuthService's own Javadoc is candid that comparing plain-text passwords is "NOT meant to be a production-grade authentication implementation." This is an accepted, explicitly documented limitation appropriate for a demo/teaching scope, but it is worth calling out clearly in a technical report so that the security trade-off is not mistaken for an oversight.

14. Conclusion

The Online Food Delivery System successfully demonstrates a complete, if intentionally scoped, ordering workflow: role-based authentication, a customer-facing menu-to-checkout journey, immutable invoice generation, and an administrator dashboard that consolidates every order into one searchable, live-updating view. The codebase achieves this with a compact set of nine classes across two packages, using no external database and only a handful of well-chosen third-party dependencies (JavaFX, ControlsFX, FormsFX).

As an academic exercise, the project meets its stated objectives well: it cleanly demonstrates abstraction, encapsulation, inheritance, and polymorphism inside a working feature set rather than as textbook snippets, and it applies sound design choices a singleton store for shared state, immutable order snapshots, and clear separation between domain classes and JavaFX UI code — that would generalise well to a larger

system. Its most notable limitations (plain-text credential storage, a hand-rolled flat-file persistence format, and a small amount of leftover scaffold/draft code) are consistent with its explicitly documented demo scope rather than being accidental oversights, and are documented in Section 13 so they can be weighed appropriately by an evaluator.

15. Recommendations

The following improvements would move the system meaningfully closer to a production-ready application, roughly in order of impact:

1. Replace plain-text password comparison in AuthService with a salted hash (e.g. BCrypt or PBKDF2), and avoid keeping credentials purely in memory so accounts survive an application restart.
2. Replace the hand-rolled pipe-delimited flat file with an embedded relational database such as SQLite (via JDBC), which would remove the manual escaping logic in Order and provide transactional writes, indexing, and safer concurrent access.
3. Add an administrator-facing menu management screen (create/edit/deactivate MenuItem entries) so the catalogue is no longer hard-coded in loadMockMenuData().
4. Introduce automated tests using the JUnit Jupiter dependency that is already declared in pom.xml but currently has no accompanying test classes — particularly

for Order.toRecordLine()/fromRecordLine() round-tripping and AuthService's registration edge cases.

5. Add order status tracking (e.g. Placed → Preparing → Out for Delivery → Delivered) as a natural extension of the existing Order model, which would let the administrator dashboard double as a basic operations tool rather than a read-only ledger.
6. Consider externalising the currently hard-coded restaurant name ("PIO Online Food Delivery Company") and menu into a configuration file, which would let the same codebase serve multiple restaurants with minimal change.

16. References

Oracle. JavaFX Documentation. Oracle Corporation. https://openjfx.io/

Apache Software Foundation. Apache JavaFx project Documentation. https://maven.apache.org/

ControlsFX. Additional UI controls for JavaFX. https://github.com/controlsfx/controlsfx

DLSC Software & Consulting. FormsFX — a framework for creating forms in JavaFX. https://github.com/dlemmermann/FormsFX

JUnit Team. JUnit 5 User Guide. https://junit.org/junit5/

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Prince (PRINCE2025EASY). FoodDevliveryApp source repository. GitHub. https://github.com/PRINCE2025EASY/FoodDevliveryApp